Name of the Student	SHAM S.
Reg. No	192325076
GitHub Link	https://github.com/Sham-2005/MLA0403-Deep_learning

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Model

COURSE NAME: Deep Learning **COURSE CODE: MLA04**
SLOT :

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

Course Outcome Weightage: 10% **Duration: 90 minutes**

Assessment Tool Weightage: 30% **Mark : 25**

Code of Conduct:

I SHAM S (192325076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Name of the student:**

Criteria	Problem Understanding & Requirement Analysis **(2)**	Model Architecture & Design **(2.5)**	Application of Deep Learning Techniques **(2.5)**	Model Development & Implementation Strategy **(2)**	Performance Analysis & Justification ***(1)***	**Total (10)**
Weightage	***20 %***	***30%***	***25%***	***15 %***	***10 %***	**100 %**
Q1						
Q2						
Q3						
Q4						
Q5						

Total						

QUESTIONS: 05 **Total Marks:25**

1. **Desing and develop a model for a** hospital has only **2,000 labeled X-ray images** for classifying images into normal and abnormal categories. A pre-trained ResNet model is available. **Design a CNN-based transfer learning model** by identifying which layers should be frozen, which layer should be replaced, and how the model should be trained.

2. An agricultural organization wants to identify **five types of plant diseases** from leaf images. Since the dataset contains limited training samples, the organization wants to reuse features efficiently. **Design a DenseNet-based classification model** and explain how dense connections can help the model reuse features during classification.

3. A university wants to predict a student's future academic performance using their **weekly attendance, assignment marks, internal assessment marks, and previous examination scores** collected over several weeks. **Design an LSTM-based model** that can process this sequential information and predict the student's performance category.

4. An organization wants to classify customer reviews as **positive, negative, or neutral**. Since the meaning of a word may depend on both the words appearing before and after it, **design a Bidirectional RNN model** for this application. Identify the major processing stages from input text to final sentiment prediction.

5. A software company wants to develop a system that translates **English sentences into Hindi** . The length of the input and output sentences may differ, and the system needs to understand the complete input sentence before generating the translated sentence. **Design an Encoder–Decoder model** and describe the role of the encoder, decoder, hidden representation, and output layer in the translation process

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding & Requirement Analysis	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding ; some requirements unclear	Poor understanding ; misinterprets problem context

Model Architecture & Design	30%	Applies advanced and relevant	Applies appropriate concepts with	Limited application of concepts	Incorrect or no application of concepts

CO4 – ASSESSMENT TOOL 1

Design and Develop Model – Deep Learning (MLA04)

Q1. CNN-Based Transfer Learning Model for X-Ray Classification

1. Problem Understanding and Requirement Analysis

A hospital has only **2,000 labeled X-ray images** divided into two classes:

- Normal
- Abnormal

Training a deep CNN from scratch with only 2,000 images may lead to **overfitting** because the dataset is relatively small. A pre-trained **ResNet** model can be used to transfer previously learned visual features such as edges, textures, shapes, and patterns.

Therefore, the proposed solution is a **ResNet-based transfer learning model** with most of the pre-trained layers initially frozen and a new classification head added for the two classes.

2. Proposed Architecture

Input X-ray Image

↓

Image Preprocessing

- Resize to 224 × 224 pixels
- Normalize pixel values
- Apply data augmentation

↓

Pre-trained ResNet Backbone

- Early convolutional layers – Frozen
- Intermediate layers – Frozen initially
- Deeper layers – Fine-tuned later

↓

Global Average Pooling

↓

Fully Connected Layer

↓

Dropout

↓

Output Layer – 2 neurons

↓

Normal / Abnormal

Architecture

```
X-Ray Image
     ↓
Resize + Normalize
     ↓
Data Augmentation
     ↓
Pre-trained ResNet
     │
     ├── Initial Layers → Frozen
     ├── Middle Layers  → Frozen
     └── Deep Layers    → Fine-tuned
     ↓
Global Average Pooling
     ↓
Dense Layer
     ↓
Dropout
     ↓
Softmax (2 Classes)
     ↓
Normal / Abnormal
```

3. Layer Freezing Strategy

The initial convolutional layers of ResNet learn general features such as:

- Edges
- Curves

- Textures
- Basic shapes

These features are useful for X-ray images, so they should remain **frozen** during the initial training stage.

The final classification layers are removed and replaced with a new classification head.

A suitable strategy is:

Stage 1 – Feature Extraction

- Freeze the complete ResNet backbone.
- Train only the new classification head.

Stage 2 – Fine-Tuning

- Unfreeze the last few ResNet blocks.
- Continue training using a small learning rate.
- Keep the early layers frozen to avoid destroying useful pre-trained features.

4. Training Strategy

The dataset can be divided approximately into:

- 70% training
- 15% validation
- 15% testing

Data augmentation can include:

- Small rotations
- Horizontal flipping where medically appropriate
- Small translations
- Zooming
- Brightness/contrast adjustments

The model can use:

- Optimizer: Adam
- Loss function: Binary Cross-Entropy or Cross-Entropy
- Learning rate: approximately 0.001 during classifier training
- Lower learning rate such as 0.0001 during fine-tuning
- Batch size: 16 or 32
- Early stopping to prevent overfitting

5. Implementation Strategy

The model can be implemented using **Python and TensorFlow/Keras or PyTorch**.

Example workflow:

```
Load X-ray Dataset
        ↓
Train/Validation/Test Split
        ↓
Preprocessing + Augmentation
        ↓
Load Pre-trained ResNet
        ↓
Freeze Backbone
        ↓
Add New Classification Head
        ↓
Train Classifier
        ↓
Unfreeze Last ResNet Blocks
        ↓
Fine-Tune
        ↓
Evaluate on Test Dataset
```

6. Performance Evaluation

The model should not be evaluated using accuracy alone because medical classification can be affected by class imbalance.

The following metrics should be considered:

- Accuracy
- Precision
- Recall/Sensitivity
- Specificity
- F1-score
- Confusion Matrix
- ROC-AUC

For a hospital application, **recall/sensitivity for abnormal cases is particularly important**, because missing an abnormal X-ray can have serious consequences.

7. Justification

Transfer learning is suitable because only 2,000 labeled images are available. ResNet provides strong pre-trained visual features, reducing the amount of training required. Freezing early layers reduces overfitting, while fine-tuning deeper layers allows the model to adapt to X-ray-specific patterns.

Q2. DenseNet-Based Plant Disease Classification

1. Problem Understanding and Requirement Analysis

An agricultural organization needs to classify leaf images into **five different plant disease categories**. The major limitation is the availability of a small number of training samples.

Training a large CNN from scratch may cause overfitting. Therefore, a **pre-trained DenseNet** model is appropriate because DenseNet uses dense connections to reuse previously learned features.

2. Proposed Architecture

Leaf Image

↓

Preprocessing and Augmentation

↓

Pre-trained DenseNet

↓

Dense Blocks

↓

Transition Layers

↓

Global Average Pooling

↓

Fully Connected Layer

↓

Softmax – 5 Classes

Architecture

```
Leaf Image
    ↓
Resize + Normalize
    ↓
Data Augmentation
    ↓
Pre-trained DenseNet
    ↓
Dense Block 1
    ↓
Transition Layer
    ↓
Dense Block 2
```

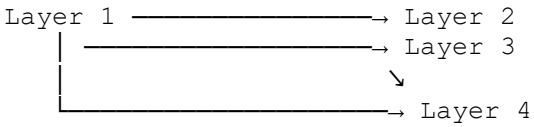
```
    ↓
Transition Layer
    ↓
Dense Block 3
    ↓
Transition Layer
    ↓
Dense Block 4
    ↓
Global Average Pooling
    ↓
Dense Layer
    ↓
Softmax (5 Classes)
```

3. Dense Connections

In a traditional CNN, a layer mainly receives information from the immediately preceding layer.

DenseNet creates connections between each layer and all subsequent layers within a dense block.

For example:



Instead of repeatedly learning the same features, later layers can directly reuse features from earlier layers.

This helps the model:

- Reuse low-level features
- Improve feature propagation
- Reduce redundant feature learning
- Improve gradient flow
- Work effectively with limited training data

4. Training Strategy

The pre-trained DenseNet backbone can initially be frozen.

Stage 1

Train the new classification layer.

Stage 2

Unfreeze selected deeper DenseNet layers and fine-tune them using a small learning rate.

Suitable techniques include:

- Data augmentation
- Dropout
- Early stopping
- Learning-rate scheduling

Possible augmentation methods:

- Rotation
- Horizontal/vertical flipping where appropriate
- Zoom
- Translation
- Brightness variation

5. Implementation Strategy

The model can be developed using:

- Python
- TensorFlow/Keras or PyTorch
- NumPy
- OpenCV/PIL
- Matplotlib
- Scikit-learn

The dataset should be divided into training, validation, and testing sets.

The final layer should contain **5 output neurons**, corresponding to the five plant disease classes.

The output activation should be **Softmax** because the task is multi-class classification.

6. Performance Evaluation

The model can be evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Per-class accuracy

A confusion matrix is particularly useful for identifying which plant diseases are frequently confused with each other.

7. Justification

DenseNet is suitable because the organization has limited training samples and needs efficient feature reuse. Dense connections allow later layers to access earlier feature representations directly. Transfer learning further reduces training requirements and improves generalization.

Q3. LSTM-Based Student Academic Performance Prediction

1. Problem Understanding and Requirement Analysis

A university wants to predict a student's future academic performance using information collected over several weeks.

The input features are:

- Weekly attendance
- Assignment marks
- Internal assessment marks
- Previous examination scores

Since the information is collected **over multiple weeks**, the data has a temporal or sequential structure.

An **LSTM (Long Short-Term Memory)** network is suitable because it can learn dependencies across time.

2. Input Representation

Suppose information is collected for 12 weeks.

Each week can be represented as:

```
[Attendance, Assignment Marks,
 Internal Marks, Previous Exam Score]
```

Therefore, the input shape becomes:

```
12 × 4
```

where:

- 12 = number of weeks
- 4 = number of features per week

3. Proposed Architecture

```
Weekly Student Data
        ↓
Data Cleaning
        ↓
Normalization
        ↓
Sequence Formation
        ↓
```

```
LSTM Layer
        ↓
Dropout
        ↓
LSTM / Dense Layer
        ↓
Fully Connected Layer
        ↓
Softmax
        ↓
Performance Category
```

For example, the performance categories may be:

- Poor
- Average
- Good
- Excellent

4. LSTM Architecture

A suitable model is:

```
Input
12 × 4
   ↓
LSTM – 64 Units
   ↓
Dropout – 0.2
   ↓
LSTM – 32 Units
   ↓
Dropout – 0.2
   ↓
Dense – 16 Units, ReLU
   ↓
Dense – 4 Units, Softmax
```

The number of output neurons should match the actual number of performance categories defined by the university.

5. Role of LSTM

LSTM contains memory mechanisms that allow it to retain important information from previous time steps.

Its main components include:

- Forget gate
- Input gate
- Cell state
- Output gate

For example, if a student's attendance decreases continuously for several weeks, the LSTM can learn that this temporal pattern may influence future performance.

6. Training Strategy

The dataset should be converted into sequences before training.

Steps:

1. Collect weekly records.
2. Clean missing values.
3. Normalize numerical features.
4. Create fixed-length sequences.
5. Split data into training, validation, and testing sets.
6. Train the LSTM.
7. Validate the model.
8. Evaluate future performance categories.

Suitable techniques:

- Adam optimizer
- Categorical Cross-Entropy loss
- Early stopping
- Dropout
- Learning-rate scheduling

7. Performance Evaluation

The model can be evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

The confusion matrix can show whether students in neighboring categories such as "Average" and "Good" are being confused.

8. Justification

LSTM is appropriate because academic data is collected over several weeks. The student's previous performance can influence future performance, creating temporal dependencies. LSTM can retain relevant historical information and use it when making the final prediction.

Q4. Bidirectional RNN for Customer Sentiment Classification

1. Problem Understanding and Requirement Analysis

An organization wants to classify customer reviews into:

- Positive
- Negative
- Neutral

The meaning of a word can depend on both the words that appear **before and after it**.

For example:

"The product is not good."

The word "good" has a different meaning because of the word "not" appearing before it.

Therefore, a **Bidirectional RNN** is suitable because it processes the sequence in both directions.

2. Processing Stages

```
Customer Review
       ↓
Text Cleaning
       ↓
Tokenization
       ↓
Vocabulary Mapping
       ↓
Padding
       ↓
Word Embedding
       ↓
Bidirectional RNN
       ↓
Dense Layer
       ↓
Softmax
       ↓
Positive / Negative / Neutral
```

3. Proposed Architecture

```
Input Text
    ↓
Tokenizer
    ↓
Sequence of Word IDs
    ↓
Padding
    ↓
Embedding Layer
    ↓
Bidirectional RNN
    ↓
Dropout
    ↓
Dense Layer
    ↓
Softmax – 3 Classes
```

A possible configuration is:

```
Embedding Layer
Vocabulary Size = Dataset dependent
Embedding Dimension = 128

        ↓

Bidirectional SimpleRNN
Units = 64

        ↓

Dropout = 0.3

        ↓

Dense
Units = 32
Activation = ReLU

        ↓

Output
Units = 3
Activation = Softmax
```

4. Bidirectional Processing

A normal RNN processes:

```
Word 1 → Word 2 → Word 3 → Word 4
```

A Bidirectional RNN processes the sequence in both directions:

```
Forward:
Word 1 → Word 2 → Word 3 → Word 4

Backward:
Word 4 → Word 3 → Word 2 → Word 1
```

The outputs from both directions are combined to create a richer representation.

This allows the model to use both previous and future context.

5. Training Strategy

The text preprocessing pipeline includes:

1. Convert reviews to lowercase.
2. Remove unnecessary symbols.
3. Tokenize text.
4. Convert tokens into integer sequences.
5. Pad sequences to a common length.
6. Pass sequences through the embedding layer.
7. Train the Bidirectional RNN.

The model can use:

- Adam optimizer
- Categorical Cross-Entropy
- Dropout
- Early stopping

6. Performance Evaluation

The model should be evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

The confusion matrix can identify whether neutral reviews are being incorrectly classified as positive or negative.

7. Justification

A Bidirectional RNN is suitable because sentiment often depends on contextual information from both sides of a word. By processing the review in forward and backward directions, the model can create a more complete representation of the sentence and improve sentiment classification.

Q5. Encoder–Decoder Model for English-to-Hindi Translation

1. Problem Understanding and Requirement Analysis

A software company wants to translate English sentences into Hindi.

The input and output sentences may have different lengths. Therefore, a simple one-to-one sequence mapping is not sufficient.

An **Encoder–Decoder architecture** is appropriate because:

- The encoder processes the complete English sentence.
- It creates a hidden representation of the input.
- The decoder uses this representation to generate the Hindi sentence one word at a time.

2. Overall Architecture

```
English Sentence
```

```
        ↓
Tokenization
        ↓
Embedding
        ↓
Encoder RNN/LSTM
        ↓
Context / Hidden Representation
        ↓
Decoder LSTM
        ↓
Dense + Softmax
        ↓
Hindi Words
        ↓
Translated Hindi Sentence
```

3. Encoder

The encoder receives the English sentence as a sequence of tokens.

For example:

```
English:
"I am going to college"
```

The sentence is converted into tokens:

```
[I, am, going, to, college]
```

The embedding layer converts these tokens into numerical vectors.

The LSTM encoder processes the sequence and generates hidden-state information representing the input sentence.

4. Hidden Representation

The encoder's final hidden state and cell state provide a representation of the input sequence.

This representation contains information about:

- Sentence structure
- Word relationships
- Meaning
- Context

It is then passed to the decoder.

5. Decoder

The decoder generates the Hindi translation sequentially.

It begins with a special start token:

```
<START>
```

Then it predicts one Hindi word at a time.

For example:

```
<START>
   ↓
मैं
   ↓
कॉलेज
   ↓
जा
   ↓
रहा
   ↓
हूँ
   ↓
<END>
```

The exact Hindi output depends on the trained dataset.

6. Proposed Architecture

```
              ENCODER
English Text
      ↓
Tokenizer
      ↓
Embedding
      ↓
LSTM Encoder
      ↓
Hidden State + Cell State
      │
      │
      ↓
              DECODER
       Hindi <START>
              ↓
        Embedding
              ↓
        LSTM Decoder
              ↓
       Dense Layer
              ↓
         Softmax
              ↓
       Hindi Word 1
              ↓
       Hindi Word 2
              ↓
            ...
              ↓
        <END>
```

7. Role of Each Component

Encoder

Reads the complete English input sentence and converts it into a meaningful internal representation.

Hidden Representation

Transfers the information learned by the encoder to the decoder.

Decoder

Generates the Hindi translation one token at a time.

Output Layer

Uses Softmax to calculate the probability of each possible Hindi vocabulary word.

The word with the highest probability can be selected during inference, depending on the decoding strategy.

8. Training Strategy

A parallel English-Hindi dataset is required.

Each training example contains:

```
English Sentence → Hindi Sentence
```

Special tokens can be added:

```
<START>
<END>
```

The preprocessing steps are:

1. Clean the English and Hindi text.
2. Tokenize both languages.
3. Create separate vocabularies.
4. Convert words into integer IDs.
5. Pad sequences.
6. Train the encoder-decoder network.
7. Validate the model.
8. Test it using unseen sentences.

During training, **teacher forcing** can be used, where the actual previous target word is supplied to the decoder while learning the next word.

9. Performance Evaluation

Translation quality can be evaluated using:

- Validation loss
- Token-level accuracy
- BLEU score

- Human evaluation

BLEU can compare generated translations with reference translations.

10. Justification

The Encoder–Decoder architecture is suitable because English and Hindi sentences can have different sequence lengths and word orders. The encoder converts the source sentence into a representation, while the decoder generates the target sentence sequentially. LSTM-based encoder-decoder models can learn dependencies across the sequence and are therefore suitable for neural machine translation.

Overall Conclusion

The five proposed models use different deep learning architectures according to the characteristics of each problem:

Question	Problem	Proposed Model	Main Reason
Q1	X-ray classification	ResNet Transfer Learning	Limited labeled images and reuse of pre-trained visual features
Q2	Plant disease classification	DenseNet	Efficient feature reuse through dense connections
Q3	Academic prediction	LSTM	Sequential weekly student data
Q4	Sentiment analysis	Bidirectional RNN	Uses context from both past and future words
Q5	English-Hindi translation	Encoder–Decoder LSTM	Handles variable-length input and output sequences

These designs satisfy the assessment's emphasis on **problem understanding, architecture and design, application of deep learning techniques, implementation strategy, and performance analysis**.

RUBRICS FOR CALCULATION:

		concepts effectively to solve the problem	minor errors		

Application of Deep Learning Techniques	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Model Development & Implementation Strategy	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Performance Analysis & Justification	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis